MapleLegends Spawn Sync Tracker: Architecture Document

Version 0.1 · October 22, 2026 · Status: Draft for review

1. Overview

The Spawn Sync Tracker is a small web application where MapleLegends players record when monsters spawn on a given map and channel, so that the community can discover which map and channel combinations spawn **in sync** with each other.

Monsters respawn on a fixed 10-second cycle, but each channel (and each map) has its own offset within that cycle. Ch1 might spawn at :12, :22, :32 and ch2 at :14, :24, :34. If a player knows that two combinations share (nearly) the same offset, they can hop between them and keep farming on the same rhythm.

Primary goals

- Let a player record a spawn observation for a map and channel in one tap.
- Aggregate observations into a reliable per-combination phase estimate with a confidence indicator.
- Answer the question: given this map and channel, which other combinations are within N milliseconds of it?
- Run entirely on free hosting tiers with no servers to maintain.

Accuracy target: observations are manual (human taps), and a tolerance of about 300 ms is acceptable. The design therefore favors simplicity over precision engineering. Reaction-time calibration and metronome-assisted tapping are optional later enhancements, not requirements.

2. Scope

In scope: manual observation capture, shared community data, phase estimation, sync search, overview visualizations, basic abuse protection.

Out of scope (v1): user accounts and logins, in-game integration of any kind (the tool is a stopwatch-style web page and never touches the game client), automated spawn detection, native mobile apps, real-time push updates.

3. Key Concepts and Assumptions

Concept	Definition
Period (P)	10,000 ms. The spawn cycle length.
Phase	`server_timestamp_ms mod 10000`. Where in the 10-second cycle a spawn occurs. Ch1 spawning at :12, :22 is phase 2000.
Circular distance	Shortest distance between two phases on the 10 s circle. 9,900 ms and 100 ms are 200 ms apart, not 9,800.
Tolerance	Maximum circular distance for two combinations to count as in sync. Default 300 ms, adjustable in the UI.
Epoch	A period during which phases are assumed stable. A server restart or similar event may reset the offsets, which starts a new epoch.
Combination	A (map, channel) pair.

Assumptions to validate early (see Section 13):

1. The period is exactly 10.000 s, with no meaningful drift.
2. Phases stay stable between restarts, and a restart changes them.
3. A consistent human bias (for example, always tapping 200 ms late) is acceptable, because it affects all combinations roughly equally.

4. System Architecture

The system is a single-page web app served from Cloudflare's edge, a stateless API running as a Cloudflare Worker, and a SQLite-compatible database (D1) in the same account.

```
Player browser (React SPA)
  |  GET  /api/time           (x5 on load: clock sync)
  |  POST /api/observations   (taps for a map + channel)
  |  GET  /api/phases, /api/sync, /api/overview
  v
Cloudflare edge
  +-- Static assets (SPA bundle)
  +-- Worker (Hono API)
        |-- input validation (Zod, shared schemas)
        |-- rate limiting + optional Turnstile check
        |-- phase math (shared TypeScript package)
        |-- short-TTL edge cache for read endpoints
        v
      D1 (SQLite): maps, epochs, observations, phase_estimates
```

Design principles

- **Server owns the truth about time.** The client syncs to server time, and the server computes the phase itself. The client's claimed phase is never trusted.
- **Store raw, derive aggregates.** Raw taps are kept so estimates can be recomputed if the algorithm changes. A small `phase_estimates` cache keeps reads cheap.
- **Bounded work per request.** Every request reads and writes a small, predictable number of rows, which keeps it inside free-tier limits.
- **Shared code.** Phase math and request schemas live in one package used by both the frontend and the Worker, so they cannot drift apart.

5. Technology Stack

Layer	Choice	Why
Language	TypeScript everywhere	One language, shared types and math between client and server.
Frontend	React + Vite SPA	Simple build, fast dev loop, large ecosystem. Svelte is an equally valid swap.
Data fetching	TanStack Query	Caching, retries, and loading states for the read endpoints.

Layer	Choice	Why
Styling	Tailwind CSS	Quick, consistent UI without a design system.
Visualization	Inline SVG (spawn ring) and CSS grid (heatmap)	No chart library needed.
API framework	Hono on Cloudflare Workers	Lightweight, typed routing, built for Workers.
Validation	Zod	Shared request and response schemas.
Database	Cloudflare D1 (SQLite)	Free tier, same account as the Worker, no cold-start pauses.
DB access	Thin repository layer over raw SQL (Drizzle optional)	Keeps queries explicit and makes a later Postgres migration easy.
Hosting	Cloudflare Pages or Workers static assets, plus Workers	Free `*.pages.dev` or `*.workers.dev` domain, HTTPS included.
Tooling	Wrangler (deploy, migrations, local dev)	Local D1 emulation and scripted migrations.
Tests	Vitest (unit and API), Playwright (optional end-to-end)	Fast tests for the circular math, which is the riskiest logic.
CI/CD	Cloudflare Git integration or GitHub Actions	Preview deploys per pull request, production on merge to `main`.

6. Repository Layout

```
spawn-sync/
  apps/
    web/            # React SPA (Vite)
    api/            # Worker (Hono), wrangler.toml, migrations/
  packages/
    shared/         # Zod schemas, constants, circular-math functions
  scripts/
    seed-maps.ts    # loads the map list into D1
  package.json      # workspace root (pnpm or npm workspaces)
```

7. Data Model

```
CREATE TABLE maps (
  id     INTEGER PRIMARY KEY,
  name   TEXT NOT NULL UNIQUE,
  region TEXT
);

CREATE TABLE epochs (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  started_at INTEGER NOT NULL,      -- unix ms
  note       TEXT
);

CREATE TABLE observations (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  map_id       INTEGER NOT NULL REFERENCES maps(id),
  channel      INTEGER NOT NULL CHECK (channel >= 1),
  server_ts_ms INTEGER NOT NULL,    -- synced timestamp of the tap
  phase_ms     INTEGER NOT NULL CHECK (phase_ms BETWEEN 0 AND 9999),
  client_id    TEXT NOT NULL,       -- anonymous random id from the browser
  epoch_id     INTEGER NOT NULL REFERENCES epochs(id),
  created_at   INTEGER NOT NULL
);
CREATE INDEX idx_obs_combo
  ON observations (map_id, channel, epoch_id, created_at DESC);

CREATE TABLE phase_estimates (
  map_id        INTEGER NOT NULL,
```

```
  channel       INTEGER NOT NULL,
  epoch_id      INTEGER NOT NULL,
  mean_phase_ms INTEGER NOT NULL,
  r             REAL NOT NULL,      -- concentration, 0 to 1
  spread_ms     INTEGER NOT NULL,   -- circular standard deviation
  n             INTEGER NOT NULL,   -- observations used
  updated_at    INTEGER NOT NULL,
  PRIMARY KEY (map_id, channel, epoch_id)
);
```

Notes

- The current epoch is the row with the highest `id` in `epochs`.
- The maximum channel number is a configuration constant rather than a hard database limit, since it is not yet confirmed.
- Old observations from past epochs can be pruned or archived later. At the expected volume, storage is not a concern.

8. Core Algorithms

All of these live in `packages/shared` and are unit tested.

8.1 Circular mean and spread

```
export const P = 10_000;
const TAU = 2 * Math.PI;

export function circMean(phases: number[]) {
  let x = 0, y = 0;
  for (const p of phases) {
    const a = (p / P) * TAU;
    x += Math.cos(a);
    y += Math.sin(a);
  }
  const n = phases.length;
  const r = Math.hypot(x, y) / n;
  const mean = ((Math.atan2(y, x) / TAU) * P + P) % P;
  const spread = r > 0 ? Math.sqrt(-2 * Math.log(r)) * (P / TAU) : P / 2;
  return { mean, r, spread, n };
}
```

```
// Signed shortest distance from b to a, in ms (range -5000..5000)
export const diff = (a: number, b: number) =>
  (((a - b + P / 2) % P) + P) % P - P / 2;
```

8.2 Estimate update (on every submission)

1. Read the most recent 50 observations for the combination in the current epoch (bounded read).
2. Compute the circular median phase, then drop any observation more than 500 ms away from it.
3. Compute `circMean` on the remaining observations.
4. Upsert the result into `phase_estimates`.

Using a rolling window of 50 means a bad batch of taps ages out, and the work per request never grows.

8.3 Confidence

Label	Rule
Low	n < 5, or spread above 250 ms
Medium	n >= 5 and spread 120 to 250 ms
High	n >= 10 and spread under 120 ms

The sync finder hides Low-confidence combinations by default, with a toggle to show them.

8.4 Sync search

Given a reference combination and a tolerance:

1. Load all `phase_estimates` rows for the current epoch. This is at most a few hundred rows.
2. For each other combination, compute `abs(diff(phase, referencePhase))`.
3. Keep those at or under the tolerance, sorted by distance and then by confidence.

8.5 Sync groups (overview)

Sort all estimates by phase and cluster neighbors whose gap is at or under the tolerance (single linkage). Handle wrap-around by joining the last and first clusters if their gap is within tolerance. Each cluster is a set of combinations that can be hopped between without losing rhythm.

8.6 Constants

Constant	Default
`P`	10000 ms
`TOLERANCE_DEFAULT`	300 ms
`WINDOW`	50 observations
`OUTLIER_MS`	500 ms
`MAX_TAPS_PER_REQUEST`	10
`MAX_CLOCK_SKEW_MS`	30,000 ms
`MAX_CHANNEL`	configurable

9. API Specification

All endpoints are under `/api`, return JSON, and are validated with shared Zod schemas.

Method and path	Purpose	Notes
`GET /api/time`	Returns `{ serverNow }` in unix ms	Never cached. Used for clock sync.
`GET /api/maps`	Lists maps and channel count	Cached for hours.
`POST /api/observations`	Records taps	Body: `{ mapId, channel, clientId, taps: [serverTsMs, ...] }`. Returns the updated estimate.

Method and path	Purpose	Notes
`GET /api/phases?mapId=`	Phase estimates for every channel on a map	Cached 30 to 60 seconds.
`GET /api/sync?mapId=&channel=&tol=`	Combinations within tolerance of the reference	Cached 30 to 60 seconds.
`GET /api/overview`	All estimates plus computed sync groups	Cached 30 to 60 seconds.
`POST /api/admin/epoch`	Starts a new epoch (server restart)	Protected by a secret token.

POST /api/observations rules

- 1 to 10 taps per request.
- Each tap timestamp must be within 30 seconds of the Worker's own clock, otherwise it is rejected.
- The server computes `phase_ms` itself from each timestamp.
- Map must exist, and channel must be between 1 and `MAX_CHANNEL`.
- Responses use standard status codes: 400 for validation errors, 429 for rate limiting.

10. Frontend Design

10.1 Pages

Route	Purpose
`/record`	Pick a map and channel, then tap a large button (or press Space) on each spawn. Shows tap count, running phase estimate, and confidence.
`/sync`	Pick a reference combination and tolerance, then view matching combinations sorted by closeness and confidence.
`/overview`	A 10-second ring with a dot per combination, plus a map by channel heatmap colored by phase. Sync groups are highlighted.

Route	Purpose
`/about`	How to measure well, what the numbers mean, known limitations.

10.2 Recording flow

1. The page syncs its clock on load.
2. The player selects a map and channel, then taps on each spawn (5 to 10 taps is plenty).
3. Taps accumulate locally. Pressing Submit sends them in one request.
4. The response shows the updated estimate for that combination, and the player can switch channel and continue.

A small UX detail goes a long way here: after a tap, show the phase in seconds (for example, `:12.4`), so players can sanity-check against what they see.

10.3 Clock sync

Because phase is computed against server time, the browser must know the offset between its clock and the server's. Five quick pings are taken and the one with the lowest round-trip time wins. A monotonic clock (`performance.now()`) is used for timing, and the sync repeats every few minutes.

```
let anchor = { perf: 0, server: 0 };

async function syncClock(samples = 5) {
  let best: { rtt: number; perf: number; server: number } | null = null;
  for (let i = 0; i < samples; i++) {
    const t0 = performance.now();
    const { serverNow } = await (await fetch('/api/time')).json();
    const t1 = performance.now();
    const rtt = t1 - t0;
    if (!best || rtt < best.rtt) {
      best = { rtt, perf: t1, server: serverNow + rtt / 2 };
    }
  }
  if (best) anchor = best;
}

export const serverTime = () => anchor.server + (performance.now() -
anchor.perf);
```

The worst-case error from this method is roughly half the round-trip time, typically a few tens of milliseconds. That is well inside the 300 ms tolerance.

10.4 State and persistence

- Server data (estimates, maps) lives in TanStack Query.
- The in-progress tap session lives in React state.
- The anonymous `client_id` (a random UUID) and last-used map and channel are kept in `localStorage`, wrapped in try/catch with sensible defaults.
- The layout is mobile-friendly, since many players will have the tracker open on a phone beside the game.

11. Backend Design

11.1 Worker structure

```
apps/api/src/
  index.ts            # Hono app, routes, middleware
  routes/
    time.ts  maps.ts  observations.ts  phases.ts  sync.ts  overview.ts
admin.ts
  db/
    repo.ts           # all SQL lives here
  lib/
    estimate.ts       # window read, outlier trim, upsert
    ratelimit.ts
```

11.2 Request lifecycle for a submission

1. Rate limit check (by `client_id` and IP).
2. Zod validation of the body.
3. Clock-skew check on each tap.
4. Look up the current epoch.
5. Insert the taps in one D1 batch (atomic).
6. Recompute and upsert the estimate (bounded to 50 rows read).
7. Return the estimate and invalidate or skip the cached read responses (a 30 to 60 second staleness is acceptable).

11.3 Caching

Read endpoints use the edge cache with short TTLs. Most traffic is people viewing the same overview, so this keeps database reads low regardless of audience size.

12. Hosting, Deployment, and Operations

12.1 Free-tier fit

Resource	Free-tier consideration
Workers	100,000 requests per day and 30 ms of CPU per request. The math here is tiny, so the CPU limit is not a concern.
D1	Daily row read and write limits apply, and from September 1, 2026 Cloudflare rejects queries once a free account exceeds them (reset at midnight UTC). Stored data is unaffected.
Static assets	Served from the edge at no cost for a small SPA.

Exact D1 numbers should be confirmed on Cloudflare's pricing page before launch, since limits can change.

Rough budget: 500 active players per day at around 30 requests each is about 15,000 requests, far below the Workers cap. Bounded queries (50-row window, cached reads, indexes on every lookup) keep D1 usage low.

12.2 Environments

Environment	Purpose
Local	`wrangler dev` with a local D1 emulation and a seeded map list.
Preview	Automatic deploy per pull request, with its own D1 database.
Production	Deployed from `main`.

12.3 Deployment steps

1. Create the D1 database and bind it in `wrangler.toml`.
2. Apply migrations with Wrangler (`migrations/0001_init.sql`), then run the map seed script.

3. Set secrets (`ADMIN_TOKEN`, optional Turnstile secret) with `wrangler secret put`.
4. Connect the Git repository so that pushes build and deploy automatically.
5. Optionally attach a custom domain. The default Cloudflare subdomain is free.

12.4 Backups and portability

- Schedule a periodic database export (for example, a GitHub Actions cron job running a Wrangler export) and store it as a build artifact or private repository file.
- Check D1's point-in-time recovery options and retention on the free plan.
- Keeping all SQL inside `db/repo.ts` and using portable SQL means a move to Supabase (Postgres) is a contained change. Note that Supabase's free tier pauses projects after a week of inactivity, which is the main reason it is the fallback rather than the default.

13. Security and Abuse Prevention

Submissions are public and anonymous, so the system must tolerate bad or careless data.

- **Server-side validation:** phase computed by the server, timestamps bounded to the server clock, strict schemas, parameterized queries only.
- **Rate limiting:** per client and per IP, using a Workers rate-limiting facility where available or a lightweight check against recent rows.
- **Optional bot protection:** Cloudflare Turnstile on the submit action if spam appears.
- **Outlier resistance:** the median-based trim and rolling window limit the damage a single bad submitter can do. Estimates also expose `n`, so thin data is visibly weak.
- **Epoch control:** only the admin token can start a new epoch in v1, so strangers cannot wipe the active data set.
- **Privacy:** no accounts, no personal data. The `client_id` is a random value, and IP addresses are used only for rate limiting and are not stored with observations.
- **Game rules:** the tracker is a manual web stopwatch. It does not read or modify the game, but it is still worth a quick read of the server's rules before publicizing it.

14. Testing and Observability

- **Unit tests (Vitest):** circular mean and difference across the wrap-around (for example, 9,900 and 100), outlier trimming, confidence labels, sync grouping with wrap-around.
- **API tests:** run against a local D1 instance, covering validation failures, clock-skew rejection, and estimate updates.

- **Optional end-to-end:** a Playwright run that taps through the record flow.
- **Observability:** Worker logs and Cloudflare's built-in analytics are enough for v1. Add counters for rejected submissions and rate-limit hits to spot abuse early.

15. Risks and Open Questions

Risk or question	Mitigation or next step
Period is not exactly 10 s, so phases drift	Run a long single-channel experiment first. Raw timestamps are stored, so drift can be fitted later.
Restarts or channel reloads change offsets	Epoch model with an admin trigger. Later, auto-flag a possible reset when several distinct clients disagree with a well-established estimate.
Unknown channel count and map list source	Make both configuration. Seed maps from a hand-curated JSON file.
Human tap noise exceeds expectations	300 ms tolerance, multi-tap averaging, outlier trimming. Optional later: metronome mode and per-user latency calibration.
Free-tier limits change	Bounded queries, caching, and a portable data layer.
Data poisoning by a malicious submitter	Rate limits, median trimming, Turnstile, and (later) client-level trust weighting.

16. Implementation Roadmap

Phase	Deliverable	Outcome
0. Validation	Static page with clock sync, tap capture, and the circular mean, no backend	Confirms the 10 s period and shows real tap noise in-game.

Phase	Deliverable	Outcome
1. Backend MVP	Worker, D1 schema and migrations, `/time`, `/maps`, `/observations`, `/phases`	Shared data between players.
2. Sync finder	`/sync` endpoint and page, confidence labels	Core goal delivered.
3. Overview	Spawn ring, heatmap, sync groups	Community-wide picture at a glance.
4. Hardening	Rate limiting, Turnstile, epochs and admin tools, backups, tests in CI	Ready to share publicly.
5. Optional	Metronome mode, latency calibration, trust weighting, PWA install	Higher precision and polish.